

Kaleb Tello

Dr. Satzhan Sitmukhambeto

CSCI-3334-02

4 May, 2026

Design Report Concurrent Task Dispatcher

Architecture Description

This project implements a simulated concurrent task dispatcher in Rust. The system models four main components: a task generator, a dispatcher queue, a worker pool, and a monitor. The purpose of the system is to simulate how tasks move through a constrained concurrent execution environment and to compare the performance of different scheduling strategies.

The task generator produces 1000 tasks at fixed 20ms intervals. Each task is classified as either an IO task or a CPU task using a deterministic 70/30 probability split. IO tasks consume 10% CPU for 200ms, while CPU tasks consume 35% CPU for 200ms.

Generated tasks are placed into a central queue, where they wait until the scheduler determines they can be dispatched. The worker pool contains up to 8 workers, each capable of processing one task at a time. A task may only be assigned to a worker if both a worker is available and the global CPU usage cap of 100% is not exceeded.

A monitor records system state every 10ms during execution. At each sample, the monitor records current simulation time, CPU usage, active worker count, and queue length. These samples are stored in memory and exported as CSV for analysis after the simulation completes.

Data Structures Used

The system uses several core Rust data structures to model task dispatching and execution.

Tasks are represented using a Task struct. Each task stores:

- task id
- task type (Io or Cpu)
- CPU cost
- creation timestamp
- start timestamp
- finish timestamp

This structure allows each task to be tracked throughout its full lifecycle.

Workers are represented using a Worker struct. Each worker stores:

- whether it is busy
- the task currently assigned
- the task's finish time

This allows the simulation to determine when workers become available again.

The dispatcher queue is implemented using VecDeque<Task>, which provides efficient push and pop operations from both ends and is well-suited for FIFO-style scheduling.

Monitoring data is stored in a Vec<LogEntry>. Each LogEntry stores:

- current time
- CPU usage
- active workers
- queue length

Completed tasks are stored in a Vec<Task> so that task-level metrics such as wait time and turnaround time can be computed after the simulation ends.

Synchronization Strategy

Although this project simulates concurrent execution, it does not use real operating system threads. Instead, concurrency is modeled through a deterministic discrete-time simulation.

The simulation advances in fixed 10ms increments. At each tick, the system performs the following steps in order:

1. generate new tasks
2. release completed tasks
3. schedule waiting tasks
4. record monitor data

This design avoids the need for mutexes, locks, or condition variables because all state changes occur in a single-threaded deterministic update loop.

A discrete-time simulation was chosen instead of real threads because it provides deterministic and repeatable behavior. Real threads would introduce operating system scheduling noise, race conditions, and nondeterministic timing, making fair comparison between FIFO and optimized scheduling much more difficult.

This approach sacrifices realism, but it improves correctness, reproducibility, and ease of debugging.

Scheduling Policy

Two scheduling strategies were implemented and compared.

The FIFO scheduler dispatches tasks strictly in arrival order. It always examines the task at the front of the queue and dispatches it when both worker and CPU constraints allow. This policy is simple and fair in arrival ordering, but it does not account for task cost.

The optimized scheduler uses a cost-aware scheduling strategy. When CPU usage is high, it prioritizes IO tasks because they consume less CPU and are easier to fit into the remaining CPU budget. When CPU usage is lower, it prioritizes CPU-heavy tasks to improve overall utilization.

FIFO prioritizes fairness and simplicity, while the optimized scheduler prioritizes efficiency and throughput.

Metrics Collected

The system records both aggregate metrics and task-level metrics.

Aggregate metrics include:

- total runtime
- makespan
- average CPU usage
- average worker utilization
- monitor sample count

Task-level metrics include:

- average wait time
- average wait time for IO tasks
- average wait time for CPU tasks
- average turnaround time
- maximum wait time

These metrics allow both schedulers to be evaluated not only by throughput, but also by responsiveness and queue efficiency.

Experiment Results

Both schedulers were tested using the same deterministic workload of 1000 tasks generated at 20ms intervals with a fixed 70/30 IO-to-CPU distribution.

In the FIFO experiment, the scheduler completed all 1000 tasks with lower CPU utilization and longer total runtime. FIFO achieved an average CPU usage of 89.65%, an average worker utilization of 5.07 active workers, and a total runtime of 39430ms.

In the optimized experiment, the scheduler also completed all 1000 tasks, but with better resource utilization and shorter runtime. The optimized scheduler achieved an average CPU usage of 96.45%, an average worker utilization of 5.46 active workers, and a total runtime of 36650ms.

The optimized scheduler also produced an average wait time of 9471.28ms and an average turnaround time of 9671.28ms. Average wait time for IO tasks was 11178.79ms, while CPU tasks averaged 5616.87ms. This shows that task prioritization improved throughput, but also changed wait-time distribution across task types.

Overall, the optimized scheduler completed the same workload faster, with better CPU usage and improved worker utilization.

One of the most important lessons from this project was that scheduling quality depends not only on fairness, but also on how efficiently available resources are used.

FIFO scheduling is easy to implement and easy to reason about, but it can waste resources when task costs vary. A simple resource-aware policy significantly improved throughput and CPU utilization without greatly increasing implementation complexity.

Another important lesson was that deterministic simulation is often more useful than real threading for controlled scheduler experiments. Although real threads would more closely resemble actual concurrent execution, they would make the system harder to debug and the results harder to reproduce.

Finally, this project demonstrated that small scheduling changes can significantly affect system performance. Even a lightweight optimization strategy produced measurable improvements in throughput, worker utilization, and runtime.